Notes for C++

Section 9 : More on Strings

String functions

You can write your own functions that manipulate strings. Strings can be passed as parameters to functions and strings can be returned from functions.

Example: check for valid input

It is often useful to check that the input to a program is valid. For example, if a program needs to read an integer value, it would be good to check that the user has actually entered an integer value.

For example, read a number and then display this type of message:

645 is an integer value
Hello is not an integer value
6$4?5 is not an integer value

We can do this by first reading the input as a string and then checking the string to see if it has invalid characters. This is done in the program below (for integer values only):

```
#include <iostream>
#include <string>
using namespace std;

int stringtoint(string str, int total);

int value, total, i;
string temp;
bool isnumber;

int main() {
  cout << "Enter an integer value ";
  cin >> temp;
  isnumber = true;
  total = temp.length();
  for (i = 0; i < total; i++) {
    if ((temp[i] < '0') || (temp[i] > '9')) {
      isnumber = false;
    }
  }
  if (isnumber) {
    value = stringtoint(temp, total);
    cout << "The integer value is " << value << endl;
  } else {
    cout << temp << " is not an integer value!\n";
  }
}

int stringtoint(string str, int total) {
```

```
// function to convert a string to an integer
  int num, i;
  num = 0;
  for (i = 0; i < total; i++) {
    num = num * 10 + str[i] - 48;
  }
  return num;
}
```

Interesting things to notice in this program:

- read the number as a string and then check for any illegal characters
- what about negative numbers like -24? Will this function work for them?
- can you write a similar function to input real (float) numbers?
- `stringtoint` has a string parameter and returns an integer value
- `stringtoint` is a useful function to convert a string to an integer

Example: a function with string parameters

Write a program that reads a name (with a one-word first name and a last name) and then inserts a middle name between the first name and the last name. Here is an example run of the program:

```
Enter first name and last name separated by a space: Bilbo Baggins
Enter a middle name: Fred
The final name is: Bilbo Fred Baggins
```

We will write a string function to insert the middle name in the right place.

```
#include <iostream>
#include <string>
using namespace std;

string insert(string str1, string str2);

int main() {
  string firstlast, middle, full;
  cout << "Enter first name and last name separated by a space: ";
  getline(cin, firstlast);
  cout << "Enter a middle name: ";
  cin >> middle;
  full = insert(firstlast, middle);
  cout << "The final name is: " << full << endl;
}

string insert(string str1, string str2) {
// insert str2 into str1 where there is a blank character
  string temp;
  int total, i;
  temp = "";
  total = str1.length();
  i = 0;
  while (str1[i] != ' ') {
    temp = temp + str1[i];
    i++;
  }
```

```
  temp = temp + " " + str2;
  while (i < total) {
    temp = temp + str1[i];
    i++;
  }
  return temp;
}
```

The algorithm for the `insert` function:

- we will build up the full name in the string called temp
- copy the first part of the name (before the space) into temp
- concatenate the middle name onto temp (which is the first name)
- copy the second part of the name (after the space) into temp
- return temp back to main

Note that the function `insert` does not use any global variables. It is a completely "stand-alone" function. This means that it is a good candidate to put in a library for use by other programmers. Actually, there are already a number of insert functions available in various libraries.

Arrays of strings

We use arrays to store many data items of the same type. We can use an array of strings to store many strings.

Example: a program to store 4 names

```
#include <iostream>
#include <string>
using namespace std;

string str[4];
int i;

int main() {
  cout << "Enter the first name ";
  getline(cin, str[0]);
  cout << "Enter the second name ";
  getline(cin, str[1]);
  cout << "Enter the third name ";
  getline(cin, str[2]);
  cout << "Enter the fourth name ";
  getline(cin, str[3]);
  for (i = 0; i < 4; i++) {
    cout << "Name [" << i << "] is " << str[i] << endl;
  }
}
```

Interesting things to notice in this program:

- how would you modify the program to store 1000 names? Do not actually do this!

Looking at individual characters in an array of strings

Sometimes we want to look at individual characters in an array of strings. For example, load the names of three New Zealand towns and display only the first letter.

Example: a program to display the first letter of a name

```
#include <iostream>
using namespace std;

string town[55];   // the maximum number of towns is 55
int total, i;

int main() {
  total = 3;    // we are looking only at 3 towns
  town[0] = "Auckland";
  town[1] = "Wellington";
  town[2] = "Christchurch";
  cout << "The first letters of the towns are:\n";
  for (i = 0; i < total; i++) {
    cout << "The first letter of " << town[i];
    cout << " is " << town[i][0] << endl;
  }
}
```

Interesting things to notice in this program:

- we could have 55 towns (the maximum) but we only have 3 (the current total)
- the code `town[i][0]` means look at character `0` in string `i`
- you can check any character in a string but be careful not to go out of bounds

2D Arrays – this is NOT an important part of the course

The ***dimension*** of an array tells us how many index numbers are required to reference an element of the array. The arrays that we usually use are one-dimensional (1D), for example:

```
float rain[12];   // this is a 1D array
```

To work with an element of this array, we need one index number, for example: `rain[i] = 9.1;`

You can set up a two-dimensional array (2D), for example:

```
char chessboard[8][8];   // this is a 2D array
```

To work with an element of this array, we need two index numbers, for example:

```
chessboard[6][2] = 'P';
```

An array of strings is actually a 2D array of characters. We use two index numbers – the first index tells us which string we are working with and the second index tells us which character we are working with. For example:

```
town[i][0]
```

Problems with ASCII codes and alphabetical order

People like to put strings (especially names) into alphabetical order. For example, many people store names and numbers on their phones in alphabetical order. Why do people like alphabetical order?

We would like to write programs that store strings in alphabetical order. Unfortunately this is harder than it would appear because computers do not work with alphabetical order – they work with ASCII code order. Computers assume that a string is "greater than" (or "less than") another string based on the ASCII codes of the characters in order from the left.

For example, if we have two strings APE and APPLE, the computer would use the following process to check which of these strings is "greater than" the other one.

Set index to zero
Check character[index] of the first string against character[index] of the second string
In this example, check A against A – they are the same – add 1 to the index and repeat...
Check P against P – they are the same – add 1 to the index and repeat...
Check E against P – code for P is greater than the code for E so APPLE is "greater than" APE

This all works fine and "alphabetically" we know APPLE comes after (is greater than) APE. But what if we compare elephant with ZEBRA? The code for e is greater than the code for Z so elephant comes after ZEBRA in ASCII code order --- but alphabetically elephant should come before ZEBRA.

WARNING – when sorting words (e.g. names) into alphabetical order, it will only work if all the words are the same case, i.e. all upper case or all lower case.

Sorting names

We have already looked at a sorting algorithm to sort numbers. We now use exactly the same ***algorithm*** (but not the same ***program***) to sort names into alphabetical order. We can use the same algorithm because alphabetical order (in a computer) is actually numeric order based on the ASCII codes. So alphabetical order (in a computer) is actually numeric order in disguise.

Example: a program to sort names

```
#include <iostream>
#include <string>
using namespace std;

int getnames();
void sortnames(int total);

// a MAXIMUM of 150 names - you can change this if you like
string name[150];

int main() {
  int i, total;
  total = getnames();
  sortnames(total);
  cout << "\nHere is the list in alphabetical order\n";
  for (i = 0; i < total; i++) {
    cout << name[i] << endl;
  }
}

int getnames() {
  int i;
```

```
  i = 0;
  cout << "Enter names - type ??? to stop\n";
  getline(cin, name[i]);
  while (name[i] != "???") {
    i++;
    getline(cin, name[i]);
  }
  return i;
}

void sortnames(int total) {
  int pass, i;
  string temp;
  for (pass = 0; pass < total; pass++) {
    for (i = 0; i < total - 1; i++) {
      if (name[i] > name[i+1]) {
        temp = name[i];
        name[i] = name[i+1];
        name[i+1] = temp;
      }
    }
  }
}
```

Interesting things to notice in this program:

- main is short and readable thanks to functions – use functions to help ***design*** your program
- note how the sorting code is basically the same as the code for sorting numbers
- I made `name` a ***global variable*** – why? Could you make it a ***local variable***?
- My program works for a MAXIMUM of 150 names. What if you want more names?
- ??? is a ***trailing value*** for names. What if someone has the name ???
- Each function has a local variable called i. These variables all work separately.